

EXPERIMENT 8

SPIT DOMAIN-SPECIFIC RAG CHATBOT

Subject: Generative Artificial Intelligence (GenAI)

Submitted by: Raj Mathuria

UID: 2023300139

Batch: PE-C

Experiment Title: Design and Evaluation of a Domain-Specific Retrieval-Augmented Generation (RAG) Chatbot for SPIT

Repository: https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Experiment_8_SPIT_RAG

1. Introduction

Large Language Models (LLMs) can generate natural-language responses to a wide variety of questions. However, they may produce incorrect or unsupported information when asked about a specific institution, organization, or domain. They may also lack access to updated or institution-specific information.

Retrieval-Augmented Generation (RAG) addresses this limitation by combining information retrieval with language generation. Instead of relying exclusively on information learned during training, a RAG system retrieves relevant information from a knowledge base and supplies it to the language model as context for answering a question.

In this experiment, a domain-specific RAG chatbot was developed for the Sardar Patel Institute of Technology (SPIT). The system uses institutional documents and website content as its knowledge base. When a user asks a question, the system retrieves potentially relevant text chunks and uses them to generate a response.

The experiment also evaluates retrieval performance for different values of k , where k represents the number of document chunks retrieved for a query. This helps assess how the size of the retrieved context affects the probability of finding a relevant source.

The project demonstrates the practical use of document processing, semantic embeddings, vector-based retrieval, and large language models to build a question-answering system for a specific domain.

2. Aim

To design, implement, and evaluate a domain-specific Retrieval-Augmented Generation (RAG) chatbot that answers questions about SPIT using information retrieved from a curated institutional knowledge base.

3. Objectives

1. Build a knowledge base from institutional PDF documents and website pages.
2. Divide source documents into manageable text chunks for efficient retrieval.
3. Generate semantic embeddings using an embedding model.
4. Retrieve relevant chunks in response to user questions.
5. Use a large language model to generate context-grounded answers.
6. Evaluate retrieval performance using different top-k values.
7. Measure retrieval latency and examine the relevance of retrieved information.
8. Identify limitations and suggest improvements for the chatbot.

4. Theoretical Background

4.1 Large Language Models

Large Language Models are neural-network-based systems trained on large amounts of text to understand and generate natural language. They can respond to questions, summarize documents, explain concepts, and perform other language-related tasks.

However, a general-purpose LLM may not know the latest institutional information or may generate unsupported answers. A domain-specific chatbot can reduce this problem by providing relevant source material at inference time.

4.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines two major operations:

- **Retrieval:** Identifying relevant information from an external knowledge base.
- **Generation:** Using a language model to formulate a response.

A typical RAG pipeline is:

1. Collect source documents.
2. Extract and clean their text.

3. Split the text into chunks.
4. Convert chunks into embeddings.
5. Index the embeddings for retrieval.
6. Receive a user query.
7. Retrieve the most relevant chunks.
8. Supply the query and retrieved context to the LLM.
9. Generate a context-grounded answer.

RAG allows the system to use external information without requiring the language model to be retrained every time the knowledge base changes.

4.3 Text Chunking

Long documents are divided into smaller units called chunks. Chunking allows the retrieval system to select relevant sections rather than providing entire documents to the LLM.

In this experiment, the configuration specifies:

- Chunk size: 450 words
- Chunk overlap: 60 words

Overlap preserves some context between consecutive chunks and reduces the likelihood of splitting related information across chunk boundaries.

4.4 Embeddings

An embedding is a numerical representation of text in a vector space. Texts with similar semantic meanings can have similar vector representations, even when they do not use exactly the same words.

The experiment uses the `nomio-embed-text:latest` embedding model through Ollama.

Embeddings allow the system to retrieve potentially relevant information based on semantic similarity rather than relying only on exact keyword matches.

4.5 Semantic Retrieval and Top-k

Semantic retrieval compares the embedding of a user query with embeddings stored in the knowledge-base index.

The system retrieves the top-k candidate chunks according to the retrieval mechanism.

For example:

- **k = 1:** Retrieve one candidate chunk.
- **k = 3:** Retrieve three candidate chunks.
- **k = 5:** Retrieve five candidate chunks.
- **k = 10:** Retrieve ten candidate chunks.

A larger k can increase the chance of retrieving a relevant source, but it may also introduce irrelevant context, increase prompt length, and make answer generation less focused.

4.6 Grounded Answer Generation

The retrieved chunks are supplied to a language model together with the user's question. The model uses the supplied context to generate an answer.

The experiment uses `llama3.2:latest` through Ollama, with a configured temperature of 0.1 and a maximum output length of 600 tokens.

A key design goal is to avoid presenting unsupported information as fact. If the retrieved context does not contain the requested information, the chatbot should communicate that limitation rather than inventing a definitive answer.

5. Knowledge Base and System Configuration

5.1 Knowledge Base

The project knowledge base was reported to contain the following sources:

Source type	Number of sources
PDF documents	101
Website pages	100
Total reported sources	201

These sources provide institutional information that can be searched when answering SPIT-related questions.

The number of source files or pages does not, by itself, guarantee that every possible question can be answered. The usefulness of an answer depends on whether the relevant information exists in the source material, has been extracted correctly, and can be retrieved successfully.

5.2 Technology Stack

Component	Technology
Application approach	Retrieval-Augmented Generation
Embedding provider	Ollama
Embedding model	nomic-embed-text:latest
Language model provider	Ollama
Language model	llama3.2:latest
Knowledge-base formats	PDF documents and website content
Text processing	Chunking with overlap
Retrieval index directory	artifacts/index
Results directory	results
Configuration	YAML
Evaluation data	evaluation/qa.jsonl

5.3 Configuration Parameters

The following parameters were used in the experiment configuration.

Parameter	Configured value
Random seed	42
Chunk size	450 words
Chunk overlap	60 words
Embedding model	nomic-embed-text:latest
LLM	llama3.2:latest
Default top-k	3
Similarity threshold	0.20
Temperature	0.1
Maximum output tokens	600
Evaluation top-k values	1, 3, 5, 10
Maximum evaluation questions	30
Output directory	results
Index directory	artifacts/index

Note: The configured maximum number of evaluation questions is 30, but the evaluation results summarized for this report contain five questions. Therefore, the reported metrics below describe the five-question evaluation, not a 30-question evaluation.

6. Methodology

The experiment follows a document-ingestion, indexing, retrieval, generation, and evaluation workflow.

6.1 Document Collection

Institutional PDF documents and website pages were collected to form the knowledge base. These sources serve as the external information repository for the chatbot.

6.2 Text Extraction and Preprocessing

Text is extracted from the source material and prepared for indexing. Preprocessing is necessary to make the content usable by the retrieval pipeline.

The quality of extracted text is important because missing, poorly formatted, or incorrectly extracted content can reduce retrieval accuracy.

6.3 Chunking

The processed text is divided into chunks of approximately 450 words with an overlap of 60 words. This produces smaller searchable units while preserving some surrounding context.

6.4 Embedding Generation

The embedding model converts each chunk into a numerical vector. These vectors represent semantic properties of the text and are used during similarity-based retrieval.

6.5 Indexing

The generated embeddings and their associated text are stored in the retrieval index. The configured index directory is `artifacts/index`.

This index makes it possible to search the knowledge base without processing every source document from scratch for each query.

6.6 Query Processing

When the user asks a question, the application processes the query and uses the retrieval system to find candidate chunks that may contain the answer.

6.7 Context Retrieval

The system retrieves the top-k chunks according to the configured retrieval procedure. The selected chunks are then provided as context for answer generation.

6.8 Answer Generation

The LLM receives the user's question and retrieved context and produces a natural-language response. The intended behavior is to provide answers grounded in the available source material and acknowledge when the necessary information cannot be found.

6.9 Evaluation

The retrieval component was evaluated with different top-k settings: 1, 3, 5, and 10. Retrieval hit rate and mean retrieval latency were examined to understand the trade-off between retrieving relevant information and retrieval speed.

7. Algorithm

Input: User query, indexed institutional knowledge base, top-k value.

Output: Context-grounded answer and retrieval results.

1. Load the system configuration.
2. Load the knowledge-base index.
3. Accept a question from the user.
4. Convert the question into an embedding using the configured embedding model.
5. Compare the query representation with indexed document representations.
6. Retrieve the top-k candidate chunks.
7. Construct a prompt containing the question and retrieved context.
8. Pass the prompt to the configured language model.
9. Generate a natural-language response.
10. Return the response to the user.
11. If the retrieved context does not support the requested information, communicate that the information could not be found.
12. During evaluation, compare retrieved results with the expected source labels and record retrieval latency.

8. Evaluation Methodology

8.1 Evaluation Dataset

The evaluation questions are stored in:

```
evaluation/qa.jsonl
```

The summarized evaluation included five questions. Three of these questions had expected-source labels available for assessing retrieval hits.

Because only three questions had expected-source labels, the hit-rate figures should be interpreted using that smaller denominator rather than all five questions.

8.2 Retrieval Hit Rate

Retrieval hit rate measures how often the retrieval system returns an expected source among the retrieved results.

The general formula is:

Hit Rate = (Number of evaluated queries with a relevant expected source retrieved / Number of queries with expected-source labels) × 100

For example, a hit rate of 66.7% over three labeled questions corresponds to approximately two successful retrievals out of three.

The precise evaluation behavior depends on how the project defines a hit and matches retrieved sources to expected-source labels.

8.3 Retrieval Latency

Retrieval latency measures the time required by the retrieval operation. Lower latency means that the retrieval stage takes less time to return its candidate chunks.

Retrieval latency does not necessarily include the time taken by the LLM to generate the final response. It should not be interpreted as total chatbot response time unless the measurement includes generation.

9. Results

9.1 Top-k Evaluation Results

The recorded evaluation results were:

Top-k value	Retrieval hit rate
1	0.0%
3	33.3%
5	33.3%
10	66.7%

The evaluation summary also reported mean retrieval latency between **15.89 ms and 16.51 ms** across the tested top-k settings.

Important: The available summary provides the overall latency range but does not establish which exact latency belongs to each top-k value. Therefore, the range is reported without assigning an unverified latency to an individual setting.

9.2 Interpretation of Top-k Results

Top-k = 1

The recorded hit rate was 0.0%. In this small evaluation, retrieving only the single highest-ranked chunk did not return an expected source for any of the labeled questions.

This suggests that relying exclusively on the top-ranked chunk may be insufficient for these questions.

Top-k = 3

The recorded hit rate increased to 33.3%. Retrieving more candidates improved the chance of including a relevant source compared with $k = 1$.

However, the majority of labeled questions still did not produce a retrieval hit at this setting.

Top-k = 5

The recorded hit rate remained at 33.3%. Increasing k from 3 to 5 did not improve the overall hit rate in the summarized evaluation.

This indicates that adding more retrieved chunks does not automatically guarantee better retrieval coverage.

Top-k = 10

The recorded hit rate increased to 66.7%, the highest among the tested configurations.

This suggests that retrieving more candidate chunks improved expected-source coverage for the small set of labeled questions. However, it does not establish that $k = 10$ is universally optimal, since a larger context can contain irrelevant chunks and may affect answer quality.

9.3 Retrieval Latency Results

Mean retrieval latency ranged from 15.89 ms to 16.51 ms across the tested settings.

The difference between the highest and lowest values in the reported range was approximately 0.62 ms.

This indicates that the recorded retrieval latency varied only slightly across the tested configurations in this evaluation. These figures relate to retrieval, not necessarily the total time needed to generate a chatbot response.

9.4 Example Query: First-Year Admission Eligibility

Query: "What are the First Year B.Tech admission eligibility criteria at SPIT?"

Observed response:

"I could not find this information in the available knowledge base. The retrieved context does not mention First Year B.Tech admission eligibility criteria at SPIT."

The retrieved chunks included unrelated material, such as an ARIIA certificate PDF and information about research processes. The reported similarity values for the retrieved chunks included approximately 0.6822, 0.6524, and 0.6442.

Interpretation: The system did not retrieve the relevant admission eligibility information for this query. Although it failed to answer the user's question, it appropriately indicated that the information was not found rather than providing an unsupported eligibility rule.

The similarity values alone do not prove that the chunks were relevant. This example shows why retrieval quality must be assessed using source relevance and correctness, not similarity scores alone.

9.5 Example Query: Documents Required for Admission

Query: "Which documents are required for admission to SPIT?"

The chatbot reported that the specific documents required for admission were not mentioned in the available context. Some retrieved content concerned older announcements from 2022 and 2021, while other retrieved material included `DATA_PRIVACY.md` and `README.md`.

Interpretation: The retrieval results were not sufficiently aligned with the question. The presence of older announcements or general project documentation does not establish the current list of admission documents.

The example highlights the importance of maintaining current institutional information and improving the relevance of retrieved chunks.

9.6 Overall Results Summary

Evaluation aspect	Observation
Knowledge base	101 PDFs and 100 website pages, as reported for the project
Number of evaluated questions	5
Questions with expected-source labels	3
Best recorded hit rate	66.7% at k = 10
Hit rate at k = 1	0.0%
Mean retrieval latency range	15.89–16.51 ms
Admission eligibility query	Relevant information was not found
Admission documents query	Retrieved content did not provide a reliable, specific document list
Grounding behavior	The observed responses acknowledged missing information rather than asserting an unsupported definitive answer

10. Interpretation and Inference

The experiment demonstrates that a RAG chatbot can combine a domain-specific knowledge base with a language model to answer institutional questions. It also demonstrates that the quality of the answer depends strongly on the quality of retrieval.

The top-k evaluation showed an increase in retrieval hit rate from 0.0% at k = 1 to 66.7% at k = 10. This suggests that, for the tested labeled questions, retrieving more candidate chunks increased the probability of including an expected source.

However, the evaluation was small: only five questions were summarized, and only three had expected-source labels. These figures are therefore preliminary and should not be generalized to all SPIT-related queries.

The admission-related examples exposed an important limitation. Despite the existence of a relatively large knowledge base, the retrieval system returned unrelated or insufficiently relevant chunks for the tested questions. This indicates that the size of the knowledge base alone does not guarantee that the correct answer will be retrieved.

The observed chatbot responses also illustrate the value of acknowledging missing information. When the retrieved context did not contain a supported answer, the chatbot stated that the information could not be found. This is preferable to presenting an unverified answer as fact, although it does not solve the underlying retrieval problem.

The retrieval latency range was narrow in the recorded evaluation. Nevertheless, the latency results should be considered separately from answer quality and total response time.

Inference: The experiment establishes a working RAG approach and provides initial evidence that increasing top-k can improve retrieval coverage. Further work is required to improve source relevance, verify answer correctness, and evaluate the system using a larger and more representative set of questions.

11. Conclusion

In this experiment, a domain-specific RAG chatbot was developed for answering questions about SPIT using institutional documents and website content. The system combines semantic embeddings, indexed text chunks, top-k retrieval, and language-model-based response generation.

The recorded evaluation compared top-k values of 1, 3, 5, and 10. The hit rate increased from 0.0% at $k = 1$ to 66.7% at $k = 10$, while the mean retrieval latency remained within the reported range of 15.89–16.51 ms.

The example queries also revealed that the system did not always retrieve information relevant to the user's question, particularly for admission eligibility and required admission documents. These results emphasize that a large knowledge base and a functioning LLM are not sufficient by themselves; reliable answers require effective retrieval and appropriate source coverage.

Overall, the experiment demonstrates the basic RAG workflow and the importance of evaluating retrieval independently from answer generation. It also identifies practical areas for improvement, including knowledge-base quality, metadata-based filtering, retrieval ranking, and broader evaluation.

12. Limitations and Future Improvements

12.1 Limitations

- 1. Small evaluation set:** Only five questions were included in the summarized results, and only three had expected-source labels.
- 2. Retrieval relevance:** The observed admission-related queries returned irrelevant or insufficiently relevant content.
- 3. Knowledge-base coverage:** The available sources did not provide a reliable answer to the tested admission questions.
- 4. Top-k trade-off:** Increasing the number of chunks may improve retrieval coverage but can also introduce irrelevant context.
- 5. Limited end-to-end measurements:** The reported latency range concerns retrieval and does not establish total response time.
- 6. Source freshness:** Older announcements may be retrieved when current information is required.

12.2 Future Improvements

1. Expand the evaluation dataset to include more questions covering admissions, academics, departments, fees, placements, examinations, and institutional policies.
2. Add expected-source labels to every evaluation question.
3. Improve document cleaning and remove duplicated or low-quality text.
4. Add metadata such as document title, publication date, category, and source URL.

5. Filter or rank sources using document category and freshness when appropriate.
6. Experiment with different chunk sizes and overlaps.
7. Compare alternative embedding models and retrieval methods.
8. Add a reranking stage to reorder retrieved chunks according to their relevance to the query.
9. Test hybrid retrieval that combines semantic similarity with keyword-based search.
10. Evaluate answer faithfulness, answer relevance, and correctness separately from retrieval hit rate.
11. Ensure that current official sources are available for questions about admissions and institutional requirements.
12. Measure end-to-end response time, including LLM generation.
13. Improve the fallback response when no reliable source is found.
14. Repeat evaluation over a larger dataset and report the number of queries and labeled sources alongside every metric.

13. Viva Questions and Answers

Q1. What is Retrieval-Augmented Generation?

RAG is a technique that combines information retrieval with language generation. It retrieves relevant information from an external knowledge base and supplies that information to a language model to help generate a grounded response.

Q2. Why is RAG useful for a domain-specific chatbot?

A domain-specific chatbot needs information about a particular organization or subject. RAG allows it to consult external documents rather than relying only on information learned during the LLM's training.

Q3. What is the role of embeddings in this experiment?

Embeddings represent text as numerical vectors. They enable the system to compare a user's question with document chunks and retrieve semantically related content.

Q4. What is chunking, and why is it necessary?

Chunking divides long documents into smaller text segments. It makes retrieval more targeted and allows the system to supply relevant sections instead of entire documents to the LLM.

Q5. Why is chunk overlap used?

Overlap preserves some text between consecutive chunks. It helps retain context when relevant information lies near a chunk boundary.

Q6. What does top-k mean?

Top-k is the number of candidate chunks returned by the retrieval system for a query. For example, $k = 3$ means that three candidate chunks are retrieved.

Q7. What happened when top-k increased in this experiment?

The recorded hit rate increased from 0.0% at $k = 1$ to 66.7% at $k = 10$. The result suggests that retrieving more candidates improved expected-source coverage in the small evaluation.

Q8. Does a higher top-k always produce a better answer?

No. A larger k can increase the chance of retrieving relevant information, but it can also introduce irrelevant context. The best value depends on the dataset and the retrieval and generation pipeline.

Q9. What is retrieval hit rate?

Retrieval hit rate measures the proportion of evaluated queries for which an expected relevant source was retrieved. In this experiment, the reported hit rates were calculated from the expected-source-labeled subset.

Q10. What does retrieval latency measure?

Retrieval latency measures the time taken by the retrieval stage to return candidate chunks. It does not necessarily include the time required by the LLM to generate the final answer.

Q11. Which embedding model was used?

The configured embedding model was `nomic-embed-text:latest`, accessed through Ollama.

Q12. Which language model was used?

The configured language model was `llama3.2:latest`, accessed through Ollama.

Q13. What happens if the answer is missing from the knowledge base?

The system should avoid inventing an unsupported answer and should communicate that the requested information could not be found in the available context.

Q14. What limitation was observed in the admission-related questions?

The retrieved chunks were not sufficiently relevant to the questions. As a result, the chatbot could not provide a reliable answer to the tested admission eligibility and document-requirement queries.

Q15. How can the chatbot be improved?

It can be improved by expanding and updating the knowledge base, improving text extraction, adding metadata filtering, testing different chunking strategies, using reranking or hybrid retrieval, and evaluating a larger labeled question set.

Q16. Why should retrieval and answer generation be evaluated separately?

A fluent answer does not necessarily mean that the correct source was retrieved. Evaluating retrieval independently helps identify whether errors originate from source selection or from the language model's use of the retrieved context.

14. Reproduction Commands

The repository was configured for use with `uv`. The following are example commands consistent with the project interface used during testing.

Install the project dependencies according to the repository's instructions and ensure that the required Ollama models are available.

Example model setup:

```
ollama pull nomic-embed-text:latest
```

```
ollama pull llama3.2:latest
```

Run the application using the repository's configured environment.

Example command to ask a question:

```
uv run main.py ask "What are the First Year B.Tech admission eligibility criteria at SPIT?"
```

Another example:

```
uv run main.py ask "Which documents are required for admission to SPIT?"
```

The project configuration identifies `evaluation/qa.jsonl` as the evaluation dataset and `[1, 3, 5, 10]` as the tested top-k values. Consult the repository's README for the exact evaluation command and any additional setup requirements.

15. Repository

GitHub Repository:

https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Experiment_8_SPIT_RAG

The repository contains the implementation and configuration for the SPIT domain-specific RAG chatbot. Refer to the repository for the actual source code, project dependencies, knowledge-base preparation procedure, and execution instructions.

16. References

1. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).
2. Ollama documentation: <https://ollama.com/>
3. SPIT institutional website: <https://www.spit.ac.in/>
4. Project repository: https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Experiment_8_SPIT_RAG

End of Report